

DOCUMENTATION

De MATLAB à Python / Spyder

Guide de transition

DÉMARRAGE

Introduction

Imports essentiels

INTERFACE SPYDER

Disposition des fenêtres

Graphes en fenêtre externe

Cellules & exécution

Aide en ligne

SYNTAXE PYTHON

Tableau comparatif

Commentaires

Boucles & conditions

Définir une fonction

TABLEAUX & MATRICES

Listes & tableaux numpy

Indexation ⚠

Matrices & produit

GRAPHIQUES

matplotlib — vue d'ensemble

Détail des fonctions

PROBABILITÉS

numpy.random

scipy.stats

DONNÉES

Importer des données

Analyse statistique

ÉQUIVALENCES MATLAB

Fonctions MATLAB → Python

GUIDE DE TRANSITION

De MATLAB à Python / Spyder

Ce guide s'adresse aux programmeurs MATLAB souhaitant utiliser Python avec l'environnement Spyder. Il couvre les différences de syntaxe, les équivalences de fonctions et l'utilisation des modules scientifiques numpy, matplotlib et scipy.

Python 3

Spyder 3+

numpy

matplotlib

scipy.stats

DÉMARRAGE

Introduction

Ce document est une référence des commandes Python usuelles, avec un accent sur les modules **numpy** et **matplotlib**, qui sont l'équivalent des boîtes à outils MATLAB pour le calcul scientifique.

Nous utiliserons la **version 3 de Python** et l'environnement de développement **Spyder**.

 Références utiles : **docs.python.org/fr/3** pour la doc officielle Python des modules [numpy](#) et [matplotlib](#).

DÉMARRAGE

Fonctionnement des import dans Python

Les imports permette d'utiliser des fonctions provenant d'autres programmes ou de librairies.

Pour réaliser un import, le programme importé doit se situer soit **le même repertoire que le document actif** s'il s'agit d'un code exterior. Les imports peut aussi définir un alias.

```
import numpy as np
import matplotlib.pyplot as plt
```

 Contrairement à MATLAB, Python ne charge aucune fonction mathématique par défaut. Sans ces imports, des appels comme **np.sin()** ou **plt.plot()** provoqueront une erreur **NameError** .

Il est aussi possible d'importer uniquement une fonction.

```
log10(100)
# Renvoie : NameError log10 is not defined

from math import log10

log10(100)
# Renvoie : 2.0
```

INTERFACE SPYDER

Disposition similaire des fenêtres

Spyder comporte trois fenêtres principales : l'éditeur, la console IPython, et la fenêtre d'aide. La console doit être de type **IPython** (et non une console Python classique) — vérifiez son nom dans l'onglet.

Mais il est cependant possible d'avoir accès à d'autres fentres comme celle de gestion des variables ou de destion des fichiers. Pour cela, il vous suffit de suivre la méthode ci-dessous

Disposition similaire à MATLAB

Pour retrouver une disposition proche de MATLAB :

1. Aller dans le menu **Window** (tout en haut)
2. Puis **Disposition Personnalisée / Mise en page MATLAB**

Disposition personnalisée

En haut à droite de chaque panneau, trois barres permettent d'accéder aux options **Déplacer**, **Détacher** et **Fermer** pour agencer les fenêtres à votre convenance.

Pour choisir quels panneaux afficher : **Window** → **Volets**.

Graphes en fenêtre externe

Par défaut les graphes s'affichent dans la console (mode *En ligne*). Pour les afficher dans une fenêtre interactive séparée (avec zoom, etc.) :

1. **Outils** → **Préférences** → **Console IPython**
2. Onglet **Graphiques**
3. Sortie : sélectionner **Automatique** ou **Tk**
4. Redémarrer les kernels (ou Spyder)



La fenêtre externe permet d'interagir avec le graphe (zoom, déplacement), équivalent aux figures MATLAB détachées.

Cellules & exécution

Spyder permet de diviser un script en **cellules**, à la manière des sections MATLAB (`%%` dans MATLAB → `###` dans Spyder).

Action	MATLAB	Spyder / Python
Délimiter une cellule	<code>%%</code>	<code>###</code>
Exécuter tout le script	<code>F5</code>	<code>F5</code>
Exécuter la cellule active	<code>Ctrl+Entrée</code>	<code>Ctrl+Entrée</code>
Exécuter et passer à la suivante	<code>Maj+Entrée</code>	<code>Maj+Entrée</code>
Exécuter la sélection	<code>F9</code>	<code>F9</code>

```
### Question 1 – calcul de la moyenne
X = np.array([1, 2, 3, 4])
print(np.mean(X))

### Question 2 – tracé
plt.plot(X)
plt.show()
```

Aide en ligne

Placez le curseur sur un nom de fonction dans l'éditeur et appuyez sur **Ctrl+I** pour afficher la documentation dans le panneau d'aide. Équivalent du `doc` MATLAB.



Pour les fonctions **matplotlib**, Ctrl+I ne fonctionne pas dans l'éditeur. Contournement : tapez le nom de

la fonction dans la *console*, puis Ctrl+I.

Les commandes d'aide classiques restent disponibles en console :

```
help(np.linspace)    # documentation complète
np.linspace?         # syntaxe courte IPython
```

SYNTAXE PYTHON

Tableau comparatif MATLAB → Python

Voici les différences de syntaxe les plus importantes pour un utilisateur MATLAB.

Concept	MATLAB	Python / numpy	Note
Fin de ligne (suppression affichage)	<code>a = 5;</code>	<code>a = 5</code>	Le <code>;</code> n'est pas nécessaire
Afficher une valeur	<code>disp(a)</code>	<code>print(a)</code>	
Commentaire ligne	<code>% commentaire</code>	<code># commentaire</code>	
Commentaire multi-lignes	<code>%{ ... %}</code>	<code>""" ... """</code>	
Puissance	<code>a^2</code>	<code>a**2</code>	
Opérateur ET logique	<code>&&</code>	<code>and</code>	
Opérateur OU logique	<code> </code>	<code>or</code>	
Opérateur XOR logique	<code>xor(a,b)</code>	<code>a ^ b</code>	
Booléens	<code>true / false</code>	<code>True / False</code>	Majuscule obligatoire
Fin de bloc (if, for, while)	<code>end</code>	indentation + :	Voir section boucles
Taille d'un vecteur	<code>size(x)</code>	<code>len(x)</code>	
Vecteur régulier	<code>a:step:b</code>	<code>np.arange(a, b, step)</code>	
Vecteur n points	<code>linspace(a,b,n)</code>	<code>np.linspace(a, b, n)</code>	
Produit matriciel	<code>A * B</code>	<code>np.dot(A,B)</code> ou <code>A @ B</code>	⚠ <code>*</code> = terme à terme en Python
Produit terme à terme	<code>A .* B</code>	<code>A * B</code>	
Premier index	<code>1</code>	<code>0</code>	⚠ Base 0 en Python
Vérifier le type	<code>isa(x, 'double')</code>	<code>isinstance(x, float)</code>	

SYNTAXE PYTHON

Commentaires

MATLAB

```
% commentaire ligne

%{
    commentaire
multi-lignes
%}
```

PYTHON

```
# commentaire ligne

"""
    commentaire
multi-lignes
"""
```

SYNTAXE PYTHON

DOCUMENTATION

De MATLAB à Python / Spyder

Guide de transition

DÉMARRAGE

Introduction

Imports essentiels

INTERFACE SPYDER

Disposition des fenêtres

Graphes en fenêtre externe

Cellules & exécution

Aide en ligne

SYNTAXE PYTHON

Tableau comparatif

Commentaires

Boucles & conditions

Définir une fonction

TABLEAUX & MATRICES

Listes & tableaux numpy

Indexation ⚠

Matrices & produit

GRAPHIQUES

matplotlib — vue d'ensemble

Détail des fonctions

PROBABILITÉS

numpy.random

scipy.stats

DONNÉES

Importer des données

Analyse statistique

ÉQUIVALENCES MATLAB

Fonctions MATLAB → Python

Boucles & conditions

En Python, les blocs sont délimités par l'**indentation** (et non par `end`). Ne pas oublier le **deux-points** `:` en fin de ligne d'en-tête.

MATLAB	PYTHON
<pre>if a > 0 disp('positif') elseif a == 0 disp('nul') else disp('négatif') end for i = 1:10 a = a + i; end while a > 5 a = a - 1; end</pre>	<pre>if a > 0: print('positif') elif a == 0: print('nul') else: print('négatif') for i in range(1, 11): a = a + i while a > 5: a = a - 1</pre>

⚠ **range(1, 11)** génère les entiers de 1 à 10 inclus (la borne supérieure est exclue). C'est l'équivalent de **1:10** en MATLAB.

Les opérateurs de comparaison disponibles :

Opérateur	Signification
==	Égal à
!=	Différent de
< / ≤	Strictement inférieur / inférieur ou égal
> / ≥	Strictement supérieur / supérieur ou égal
and / or / ^	ET / OU / XOR logiques

SYNTAXE PYTHON

Définir une fonction

MATLAB	PYTHON
<pre>function y = DensGauss(x,m,s2) if s2 <= 0 disp('erreur') else y = exp(-(x-m)^2/s2) ... /sqrt(2*pi*s2); end end</pre>	<pre>def DensGauss(x, m, sigma2): if sigma2 <= 0: print('erreur : variance ≤ 0') else: y = np.exp(-(x-m)**2/sigma2) y /= np.sqrt(2*np.pi*sigma2) return y</pre>

La commande `return` quitte la fonction et renvoie la valeur. Si elle est absente, la fonction renvoie `None` .

TABLEAUX & MATRICES

Listes & tableaux numpy

Python dispose d'un type `list` natif, mais pour le calcul scientifique on préfère les tableaux `ndarray` de numpy, qui permettent des opérations vectorisées.

```
X = np.array([0, 1, 2])
Y = np.array([3, 4, 5])
```

```
print(X + 2)      # [2 3 4]      - opération scalaire
print(X + Y)      # [3 5 7]      - addition terme à terme
print(X * Y)      # [0 4 10]     - produit terme à terme
print(Y**2)       # [9 16 25]    - puissance terme à terme
print(X**Y)       # [0  1 32]    - 0^3, 1^4, 2^5
```

Fonctions utiles sur les tableaux 1D

Fonction	MATLAB équivalent	Description
len(X)	length(X)	Longueur du tableau
np.sort(X)	sort(X)	Tri croissant (retourne un tableau)
np.sum(X)	sum(X)	Somme de tous les éléments
np.cumsum(X)	cumsum(X)	Sommes cumulées
np.mean(X)	mean(X)	Moyenne
np.arange(a, b, s)	a:s:b	Vecteur espacé de s (borne sup exclue)
np.linspace(a, b, n)	linspace(a,b,n)	n points entre a et b inclus
range(a, b+1)	a:b	Entiers de a à b

Appliquer une fonction à un tableau

Les fonctions numpy s'appliquent directement à un tableau (contrairement aux fonctions du module `math`) :

```
import math
X = [0, 1, 2]

math.sin(X)      # ✗ TypeError - math ne gère pas les listes
np.sin(X)        # ✓ array([0., 0.841, 0.909])
np.vectorize(math.sin)(X) # ✓ alternative pour fonctions non-numpy
[math.sin(x) for x in X]  # ✓ compréhension de liste
```

TABLEAUX & MATRICES

Indexation ⚠

⚠ **Point critique** : Python est en **base 0** (le premier élément est à l'indice 0), alors que MATLAB est en base 1. C'est la source d'erreurs la plus fréquente lors de la transition.

MATLAB — BASE 1

```
A = [10, 20, 30, 40]
A(1) % → 10 (premier)
A(end) % → 40 (dernier)
A(2:3) % → [20, 30]
```

PYTHON / NUMPY — BASE 0

```
A = np.array([10, 20, 30, 40])
A[0]      # → 10 (premier)
A[-1]     # → 40 (dernier)
A[1:3]    # → [20, 30] (indice 3 exclu)
```

ℹ La syntaxe de *slicing* `A[i:j]` retourne les éléments d'indice **i inclus** à **j exclu**. Ainsi `A[1:3]` retourne les éléments 1 et 2.

TABLEAUX & MATRICES

Matrices & produit matriciel

```
A = np.array([[1,2],[3,4],[5,6]])
```

```
A[2, 0]      # → 5           (ligne 2, col 0)
A[:, 0]      # → [1,3,5]    (toute la colonne 0)
A[2, :]      # → [5,6]      (toute la ligne 2)

np.sum(A)     # → 21        (tous les éléments)
np.sum(A, axis=0) # → [9,12] (somme par colonne)
```

⚠ En Python, l'opérateur `*` effectue une multiplication **terme à terme**. Pour le vrai produit matriciel, utilisez `np.dot(A, B)` ou l'opérateur `@` (Python 3.5+).

MATLAB

```
A * B      % produit matriciel
A .* B     % terme à terme
```

PYTHON

```
np.dot(A, B) # produit matriciel
A @ B        # idem, syntaxe moderne
A * B        # terme à terme seulement
```

💡 L'opérateur `@` a été introduit en Python 3.5 spécifiquement pour le produit matriciel. Il est équivalent à `np.dot()` et plus lisible pour les habitués de MATLAB.

GRAPHIQUES

matplotlib — vue d'ensemble

Le module `matplotlib.pyplot` est l'équivalent des fonctions graphiques de MATLAB.

Fonctions de tracé

Fonction Python	MATLAB équivalent	Utilisation
<code>plt.plot(X, Y)</code>	<code>plot(X, Y)</code>	Courbe reliant des points
<code>plt.step(X, Y)</code>	<code>stairs(X, Y)</code>	Fonction en escalier
<code>plt.stem(X, Y)</code>	<code>stem(X, Y)</code>	Diagramme en bâton
<code>plt.hist(X, bins=c)</code>	<code>hist(X, c)</code>	Histogramme
<code>plt.scatter(X, Y)</code>	<code>scatter(X, Y)</code>	Nuage de points

Fonctions d'annotation et de mise en forme

Fonction Python	MATLAB équivalent	Utilisation
<code>plt.figure()</code>	<code>figure()</code>	Créer/sélectionner une figure
<code>plt.show()</code>	—	Afficher le diagramme
<code>plt.close('all')</code>	<code>close all</code>	Fermer toutes les figures
<code>plt.subplot(m,n,k)</code>	<code>subplot(m,n,k)</code>	Subdiviser une figure
<code>plt.title(' ... ')</code>	<code>title(' ... ')</code>	Titre du graphe
<code>plt.xlabel / ylabel</code>	<code>xlabel / ylabel</code>	Légendes des axes
<code>plt.xlim / ylim</code>	<code>xlim / ylim</code>	Bornes des axes
<code>plt.legend([...])</code>	<code>legend(...)</code>	Légende des courbes
<code>plt.grid()</code>	<code>grid on</code>	Grille

ℹ Prenez l'habitude d'annoter systématiquement avec `plt.title`, `plt.xlabel`, `plt.ylabel` et `plt.legend`. Sans ces annotations, les graphes perdent leur sens en dehors de leur contexte.

Détail des fonctions graphiques

Exemple complet

```
plt.figure()
X = np.linspace(-2, 2, 1000)
plt.plot(X, np.exp(X), color='b', label='exp(x)')
plt.plot(X, 1+X+X**2/2, color='r', ls='dashed', label='1+x+x²/2')
plt.grid()
plt.ylim(-2, 8)
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.title('Approximation polynomiale de exp(x)')
plt.show()
```

plt.subplot — figures multiples

```
plt.figure()
for k in range(1, 7):
    plt.subplot(2, 3, k)          # grille 2×3, case k
    plt.title('Diagramme ' + str(k))
plt.show()
```

plt.plot — styles de courbe

```
plt.plot(X, Y, color='g')          # vert
plt.plot(X, Y, color='r', ls='dashed') # rouge pointillé
plt.plot(X, Y, color='b', ls='dotted') # bleu tirets
```

plt.hist — histogramme normalisé

```
plt.hist(X, bins=10, density=True, color='c')
# density=True → aire totale = 1 (comme une densité de probabilité)
# sans density=True → hauteur = nombre de valeurs par classe
```

plt.stem — couleurs

⚠ L'option `color=` ne fonctionne pas avec `plt.stem`. Utiliser `markerfmt` à la place : `'bD'` pour un diamant bleu, `'gs'` pour un carré vert, `'ro'` pour un cercle rouge.

```
plt.stem(X, Y, markerfmt='bD')    # marqueurs diamant bleus
```

numpy.random — variables aléatoires uniformes

```
# Tableau de s valeurs aléatoires uniformes sur [0, 1[
np.random.random_sample(s)

# Exemples
np.random.random_sample(10)      # vecteur de 10 valeurs
np.random.random_sample((3, 4))  # matrice 3×4
```

scipy.stats — lois de probabilité

Le module `scipy.stats` fournit un grand nombre de lois de probabilité sous forme d'instances des classes `scs.rv_discrete` et `scs.rv_continuous` .

Lois discrètes

Fonction	Description
<code>scs.randint(a, b)</code>	Uniforme sur {a, ..., b-1}
<code>scs.bernoulli(p)</code>	Bernoulli de paramètre p
<code>scs.binom(n, p)</code>	Binomiale (n, p)
<code>scs.geom(p)</code>	Géométrique de paramètre p (support {1, 2, ...})
<code>scs.poisson(λ)</code>	Poisson de paramètre λ
<code>scs.rv_discrete(values=(xk,pk))</code>	Loi discrète personnalisée

Lois continues

Toutes les lois continues acceptent les arguments optionnels `loc= ...` (translation) et `scale= ...` (dilatation).

Fonction	Description
<code>scs.uniform()</code>	Uniforme sur [0, 1[
<code>scs.norm()</code>	Normale centrée réduite N(0,1)
<code>scs.expon()</code>	Exponentielle de paramètre 1
<code>scs.gamma(α)</code>	Loi gamma (α, 1)
<code>scs.cauchy()</code>	Loi de Cauchy
<code>scs.pareto(b)</code>	Loi de Pareto de paramètre b

Méthodes disponibles sur toute loi

Méthode	Description
<code>.rvs(s)</code>	Échantillon de taille s
<code>.cdf(x)</code>	Fonction de répartition : P(X ≤ x)
<code>.sf(x)</code>	Fonction de survie : P(X > x)
<code>.pmf(k)</code>	Masse en k (lois discrètes seulement)
<code>.pdf(x)</code>	Densité en x (lois continues seulement)
<code>.ppf(q)</code>	Quantile d'ordre q (inverse de la cdf)
<code>.isf(q)</code>	Équivalent à ppf(1-q)
<code>.interval(α)</code>	Intervalle de confiance de niveau α

Exemples d'utilisation

```
# Les trois formulations suivantes sont équivalentes :
scs.poisson(1).pmf(x)
P = scs.poisson(1); P.pmf(x)
scs.poisson.pmf(x, 1)

# Échantillon de taille k de la loi binomiale (n, p) :
scs.binom(n, p).rvs(k)
scs.binom.rvs(n, p, size=k) # taille via mot-clé size=

# Normale N(2, 9) → loc=2, scale=3 (écart-type)
scs.norm(loc=2, scale=3).rvs(100)
```

Importer des données

```
# Charger un fichier texte (valeurs séparées par espaces)
X = np.loadtxt('donnees.txt')

# Si le fichier contient deux lignes, décomposer en deux vecteurs :
[X, Y] = np.loadtxt('donnees.txt')
```



Le fichier doit se trouver dans le **répertoire de travail de la console**. Dans Spyder : clic droit sur l'onglet du fichier de code → *Définir le répertoire de travail de la console*.

Analyse statistique d'échantillons

Fonctions numpy

Fonction Python	MATLAB équivalent	Description
<code>np.mean(X)</code>	<code>mean(X)</code>	Moyenne empirique
<code>np.var(X)</code>	<code>var(X)</code>	Variance empirique
<code>np.std(X)</code>	<code>std(X)</code>	Écart-type empirique
<code>np.cov(X)</code>	<code>cov(X)</code>	Matrice de covariance
<code>np.median(X)</code>	<code>median(X)</code>	Médiane
<code>np.quantile(X, 0.7)</code>	<code>quantile(X, 0.7)</code>	Quantile
<code>np.min(X)</code> / <code>np.max(X)</code>	<code>min(X)</code> / <code>max(X)</code>	Minimum / maximum
<code>np.average(X, weights=w)</code>	—	Moyenne pondérée

Fonctions scipy.stats supplémentaires

Fonction	Description
<code>scs.skew(X)</code>	Coefficient d'asymétrie
<code>scs.kurtosis(X)</code>	Coefficient d'aplatissement
<code>scs.moment(X, k)</code>	Moment empirique d'ordre k
<code>scs.gmean(X)</code>	Moyenne géométrique
<code>scs.tmean(X)</code> / <code>scs.tvar(X)</code>	Moyenne / variance tronquées

Fonctions MATLAB → Python

Récapitulatif des fonctions présentes en MATLAB mais qui ont un équivalent différent en Python.

MATLAB	Python	Notes
<code>pause(t)</code>	<code>time.sleep(t)</code>	Nécessite <code>import time</code>
<code>tic</code> / <code>toc</code>	<code>t0=time.time()</code> ... <code>t1=time.time()</code>	Différence t1-t0 = durée écoulée
<code>clear all</code>	<code>%reset</code>	À taper directement dans la console IPython
<code>clear <var></code>	<code>%reset_selective <var></code>	Efface une variable spécifique

DOCUMENTATION

De MATLAB à Python / Spyder

Guide de transition

DÉMARRAGE

Introduction

Imports essentiels

INTERFACE SPYDER

Disposition des fenêtres

Graphes en fenêtre externe

Cellules & exécution

Aide en ligne

SYNTAXE PYTHON

Tableau comparatif

Commentaires

Boucles & conditions

Définir une fonction

TABLEAUX & MATRICES

Listes & tableaux numpy

Indexation ⚠

Matrices & produit

GRAPHIQUES

matplotlib — vue d'ensemble

Détail des fonctions

PROBABILITÉS

numpy.random

scipy.stats

DONNÉES

Importer des données

Analyse statistique

ÉQUIVALENCES MATLAB

Fonctions MATLAB → Python

MATLAB	Python	Notes
<code>fprintf(...)</code>	<code>print(...)</code>	
<code>size(x)</code>	<code>len(x)</code>	Pour les tableaux 1D ; <code>np.shape(x)</code> pour les dimensions
<code>logical</code>	<code>bool</code>	Valeurs : <code>True</code> / <code>False</code>
<code>end</code> (fin de bloc)	<code>:</code> + <code>indentation</code>	L'indentation remplace end
<code>;</code> (suppression affichage)	<code>-</code>	Inutile : les variables ne s'affichent que via <code>print()</code>

Exemple complet : mesure de temps

```
import time

a = 0
t0 = time.time()           # équivalent de tic

for idx in range(1000):
    a = a + 1

t1 = time.time()           # équivalent de toc
time.sleep(1)              # équivalent de pause(1)
t2 = time.time()

print(t1 - t0)              # durée de la boucle
print(t2 - t1)              # durée du sleep (~1 s)
```